621. Task Scheduler Medium

Acceptance: 62.5% | LeetCode

You are given an array of CPU `tasks`, each labeled with a letter from A to Z, and a number `n`. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of **at least** `n` intervals between two tasks with the same label.

Return the **minimum** number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- `1 <= tasks.length <= 104`
- `tasks[i]` is an uppercase English letter.
- `0 <= n <= 100`

Claude Opus Solution

Task Scheduler — Complete Solution Guide

1. Intuition

The **key insight** is that the schedule's length is **bottlenecked by the most frequent task(s)**.

Think of it this way: if task `A` appears `max_count` times, and each `A` needs `n` gaps after it, then `A` alone forces a rigid skeleton:

```
A _ _ ... _  A _ _ ... _  A
└── n gaps ──┘
```

This skeleton has `(max_count - 1)` complete "cycles" of length `(n + 1)`, plus the final `A`. We then try to **fill the gaps** with other tasks. The cooling constraint is automatically satisfied because each gap is exactly `n` wide.

Two scenarios: - **Tasks don't fill the gaps** → we're forced to **idle**, and the answer is the skeleton size: `(max_count - 1) * (n + 1) + num_most_frequent`. - **Tasks overflow the gaps** → no idle needed, we just pack everything tightly, so the answer is simply `len(tasks)`.

The answer is the **maximum** of these two.

---

2. Approach (Pseudocode)

```
1. Count frequency of each task.
2. max_count = highest frequency.
3. num_most_frequent = how many tasks share that max frequency.
   (These tail tasks each add 1 to the final interval block.)
4. skeleton = (max_count - 1) * (n + 1) + num_most_frequent
5. return max(skeleton, len(tasks))
```

Why `num_most_frequent`?

If multiple tasks (say A and B) both occur `max_count` times, the last cycle must hold both:

```
A B _ ... A B    ← the final "A B" are the tail
```

So we add 1 for each task tied at the maximum.

Why `max(..., len(tasks))`?

The skeleton formula can **underestimate** when there are many distinct tasks that overflow the gaps. Example: `AAABBBCDEFG`, `n=2`. Skeleton = `2*3 + 2 = 8`, but there are 11 tasks → answer is 11 (no idle possible).

---

3. Python Solution

```python
from collections import Counter
from typing import List


class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        # Count occurrences of each task label.
        counts = Counter(tasks)

        # The most frequent task determines the rigid skeleton.
        max_count = max(counts.values())

        # How many tasks are tied for the maximum frequency.
        # Each tied task contributes to the "tail" of the last cycle.
        num_most_frequent = sum(1 for c in counts.values() if c == max_count)

        # ---- Build the skeleton length ----
        # (max_count - 1) full cycles, each of width (n + 1):
        #   one slot for the frequent task + n slots for the cooldown gap.
        # Then add the final block holding all the tied-max tasks.
        #
        # Example: AAABBB, n=2
        #   max_count = 3, num_most_frequent = 2
        #   skeleton = (3-1)*(2+1) + 2 = 6 + 2 = 8  ✓
        skeleton = (max_count - 1) * (n + 1) + num_most_frequent

        # If we have so many distinct tasks that they overflow every gap,
        # there is no idle time and the answer is just the task count.
        # Otherwise the skeleton (with idles) dominates.
        return max(skeleton, len(tasks))
```

---

4. Complexity Analysis

Metric	Complexity	Explanation
Time	`O(L)`	`L = len(tasks)`. Counting is `O(L)`; finding max and counting ties iterates over at most 26 distinct labels (`O(1)` since only A–Z).
Space	`O(1)`	The `Counter` holds at most 26 keys regardless of input size. (Treated as `O(26) = O(1)`.)

This is **optimal** — we can't do better than reading all tasks once.

---

5. Alternative Approaches

A) Greedy Heap Simulation (solution2.py)

Use a max-heap of frequencies. In each round of `n+1` slots, pop the most frequent available tasks, decrement, and re-add after the cooldown.

```python
import heapq
from collections import Counter

class Solution:
    def leastInterval(self, tasks, n):
        heap = [-c for c in Counter(tasks).values()]
        heapq.heapify(heap)
        time = 0
        while heap:
```

```
            temp = []
            # Process one full cooldown window of size (n+1).
            for _ in range(n + 1):
                if heap:
                    cnt = heapq.heappop(heap) + 1   # one occurrence used
                    if cnt < 0:                     # still remaining
                        temp.append(cnt)
                if not heap and not temp:           # everything done
                    break
                time += 1                           # advance clock (work or idle)
            for c in temp:
                heapq.heappush(heap, c)
        return time
```

- **Time:** `O(L)` total work spread across cycles (heap ops are `O(log 26) = O(1)`).
- **Space:** `O(1)`.
- **Pro:** Intuitive, mirrors actual scheduling; can be extended if you must output the actual sequence.
- **Con:** More code and slightly slower constant factor than the math formula.

B) Queue-based Simulation

Maintain a ready queue (sorted by frequency) and a "cooling" queue storing `(remaining_count, ready_time)`. Step the clock one tick at a time. Useful when the problem also asks for the exact schedule, but it's `O(total_time)` which can be larger than `O(L)`.

🎯 **Recommendation**

Use the **math formula** (Section 3) — it's the cleanest, fastest, and demonstrates the deepest understanding. Keep the **heap simulation** in your back pocket for follow-ups like "*print the actual schedule*" where the formula alone isn't enough.

Codex Solution

1. Intuition

The main difficulty is deciding when CPU must be idle.

The key insight:

> The tasks that appear the most frequently determine the minimum possible schedule length.

Suppose the most frequent task is `A`, appearing `max_freq` times.

For example:

```
A _ _ A _ _ A
```

If `n = 2`, then between every two `A`s, we need at least `2` intervals.

So we can think of arranging the most frequent task first:

```
A [gap] A [gap] A
```

There are `max_freq - 1` gaps between the `A`s.

Each gap must have length at least `n`.

Equivalently, each block has size `n + 1`:

```
A _ _ | A _ _ | A
```

The last `A` does not need a full cooling block after it.

So the base formula is:

```
(max_freq - 1) * (n + 1) + 1
```

But if multiple tasks have the same maximum frequency, they all occupy the final positions.

Example:

```
tasks = A A A B B B, n = 2
```

Both `A` and `B` appear `3` times.

Schedule shape:

```
A B idle | A B idle | A B
```

Here:

```
max_freq = 3
num_max_freq_tasks = 2
```

So the formula becomes:

```
(max_freq − 1) ∗ (n + 1) + num_max_freq_tasks
```

However, sometimes there are enough other tasks to fill all idle slots. In that case, the answer is simply `len(tasks)`.

Therefore:

```
answer = max(len(tasks), (max_freq − 1) ∗ (n + 1) + num_max_freq_tasks)
```

---

2. Approach

Pseudocode

```
count frequency of each task

max_freq = maximum frequency among all tasks

num_max_freq_tasks = number of tasks whose frequency equals max_freq

minimum_schedule_from_cooling =
    (max_freq − 1) ∗ (n + 1) + num_max_freq_tasks

answer = max(total number of tasks, minimum_schedule_from_cooling)

return answer
```

---

3. Python Solution

```python
from collections import Counter
from typing import List


class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        # Count how many times each task appears.
        #
        # Example:
        # tasks = ["A", "A", "A", "B", "B", "B"]
        # frequencies = {"A": 3, "B": 3}
        frequencies = Counter(tasks)

        # Find the highest frequency of any task.
        #
        # This task, or these tasks, will decide the minimum number
        # of cooling blocks needed.
        max_freq = max(frequencies.values())

        # Count how many tasks have this maximum frequency.
        #
        # Example:
        # A appears 3 times, B appears 3 times
        # max_freq = 3
        # num_max_freq_tasks = 2
        num_max_freq_tasks = 0
        for count in frequencies.values():
            if count == max_freq:
                num_max_freq_tasks += 1

        # Build the theoretical minimum schedule based on the most frequent tasks.
        #
        # There are max_freq − 1 full blocks.
```

```
        #
        # Each block has length n + 1:
        # one slot for the frequent task, and n cooling slots after it.
        #
        # Then we add num_max_freq_tasks because the final group contains
        # all tasks tied for maximum frequency.
        #
        # Example:
        # tasks = A A A B B B, n = 2
        #
        # max_freq = 3
        # num_max_freq_tasks = 2
        #
        # Schedule shape:
        # A B idle | A B idle | A B
        #
        # Formula:
        # (3 - 1) * (2 + 1) + 2 = 8
        intervals_needed_by_cooling = (max_freq - 1) * (n + 1) + num_max_freq_tasks

        # We can never take fewer intervals than the number of tasks,
        # because each interval can execute at most one task.
        #
        # If there are enough other tasks to fill all idle slots,
        # then the answer is simply len(tasks).
        return max(len(tasks), intervals_needed_by_cooling)
```

---

4. Complexity Analysis

Let:

```
m = len(tasks)
```

There are only uppercase English letters, so at most `26` distinct task types.

Time Complexity

```
O(m)
```

Explanation:

- Counting task frequencies takes `O(m)`.
- Scanning the frequency values takes `O(26)`, which is constant.
- Overall complexity is `O(m)`.

Space Complexity

```
O(1)
```

Explanation:

- The frequency map stores at most `26` task types.
- Since the alphabet size is fixed, the space usage is constant.

If the task labels were not limited to uppercase English letters, the space complexity would be `O(k)`, where `k` is the number of distinct task types.

---

5. Alternative Approaches

Alternative 1: Max Heap Simulation

We can simulate the CPU scheduling process using a max heap.

At each cycle of length `n + 1`, choose up to `n + 1` most frequent remaining tasks.

This works because we want to use the most frequent tasks first to avoid future idle time.

High-level idea:

```
count task frequencies

put all frequencies into a max heap
```

```
while heap is not empty:
    run up to n + 1 different tasks
    reduce their counts
    push unfinished tasks back into heap
    add time used or full cycle length depending on whether tasks remain
```

This approach is intuitive but more complex than the math solution.

Time complexity:

```
O(m log 26) = O(m)
```

Space complexity:

```
O(26) = O(1)
```

Example implementation:

```python
from collections import Counter
import heapq
from typing import List


class SolutionHeap:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        # If no cooldown is needed, we can execute tasks back-to-back.
        if n == 0:
            return len(tasks)

        # Count each task.
        frequencies = Counter(tasks)

        # Python has a min heap, so store negative counts to simulate a max heap.
        heap = [-count for count in frequencies.values()]
        heapq.heapify(heap)

        time = 0

        # Continue until all tasks are executed.
        while heap:
            # Store tasks used in this cycle that still have remaining count.
            used_tasks = []

            # A cycle has length n + 1.
            cycle_length = n + 1

            # Try to execute up to n + 1 different tasks.
            for _ in range(cycle_length):
                if heap:
                    # Pick the currently most frequent available task.
                    count = heapq.heappop(heap)

                    # Execute it once.
                    count += 1  # Since count is negative, +1 means one fewer remaining.

                    # If this task still remains, save it to push back after cycle.
                    if count < 0:
                        used_tasks.append(count)

                # Whether we executed a task or idled, one interval passes.
                time += 1

                # Optimization:
                # If no tasks are currently waiting in heap and no used tasks remain,
                # we are completely done and should not count extra idle intervals.
                if not heap and not used_tasks:
                    return time

            # After one full cooldown cycle, tasks used in this cycle become available again.
            for count in used_tasks:
                heapq.heappush(heap, count)
```

```
        return time
```

Alternative 2: Queue with Cooldown

Another simulation approach is:

- Use a max heap for available tasks.
- Use a queue for tasks cooling down.
- Each queued task stores the time when it becomes available again.

This is useful for problems where task durations or cooldown rules are more complicated.

For this LeetCode problem, the mathematical counting solution is the cleanest and most efficient.